Contents

SYSTEM DESIGN CASE STUDY

Payment System Interface Modernization on AWS

Prepared by:

Amit Kulkarni [LinkedIn] [Github]

Introduction & Executive Context

As traditional financial institutions navigate the complexities of digital transformation, legacy monolithic core banking systems face unprecedented operational strain from the explosive growth of high-velocity mobile and open-banking application channels.

This technical case study provides a rigorous, senior-level evaluation of the official AWS Reference Architecture for Payment System Interface Modernization. By exploring the critical intersections of elastic container orchestration, low-latency microservices, hybrid network topologies, and stringent financial regulatory compliances (such as PCI-DSS and SOC 2), this document bridges the gap between theoretical reference models and production-ready enterprise execution.

Rather than accepting vendor blueprints at face value, this analysis deconstructs hidden operational risks—specifically addressing synchronous hybrid runtime dependencies, shared-database anti-patterns, and cross-Availability Zone data transit overheads—to deliver actionable, prescriptive engineering remediations for building an institutional-grade, highly available payment infrastructure.

Executive Summary

This architecture provides a scalable, cloud-native blueprint for traditional financial institutions trapped by legacy constraints. It enables banks to decouple user-facing services from rigid on-premise infrastructure, accelerating feature deployment while maintaining institutional-grade security.

The Problem Statement

1. Legacy Monolithic Bottlenecks

Traditional core banking applications operate as monolithic, on-premise systems tightly bound to legacy databases and mainframes. Because the entire system scales as a single unit, spikes in mobile or digital channel traffic directly strain core banking layers. This tightly coupled architecture introduces significant operational risk, causes severe performance degradation during peak hours, and severely slows down release cycles for new customer features.

2. The Microservices Modernization Challenge

To stay competitive, financial institutions must transition toward containerized microservices to decouple their front-end interfaces from backend accounting. However, breaking down a financial monolith introduces complex challenges:

- Ensuring reliable, real-time synchronization between elastic cloud applications and rigid, on-premise core systems.
- Maintaining low-latency message streaming across transient transaction states (payment requests, balance checks, and merchant onboarding).
- Managing compute resources dynamically to handle sudden transactional spikes without over-provisioning infrastructure.

3. Strict Security & Regulatory Constraints

Financial systems operate under strict regulatory compliance mandates (such as **PCI-DSS** and **SOC 2**). Transitioning workloads to a public cloud introduces significant compliance hurdles:

- **Perimeter Defense:** Protecting customer-facing endpoints against distributed denial-of-service (DDoS) actions, automated bot injections, and application-layer threats.

- **Cryptographic Isolation:** Securing high-value transactional data at rest and in transit, requiring cloud services to integrate seamlessly with dedicated, on-premise Hardware Security Modules (HSMs).
- **Data Privacy:** Isolating internal microservice communications and caching layers from the public internet using private networks and strict access management protocols.

4. Operational Invisibility

Legacy systems often suffer from fragmented logging and blind spots across decoupled infrastructure. In a modernized, distributed cloud environment, the lack of centralized tracing makes it exceptionally difficult to debug failed payment lifecycles, monitor latency bottlenecks across API boundaries, or maintain an audit trail for regulatory compliance.

Transaction Data-Flow Guide

This technical guide traces the end-to-end lifecycle of a retail transaction through the modernized architecture, moving from initial client ingress to final backend reconciliation.

Step 1: Perimeter Defense & Ingress (Anchor 1 & 4)

- **Action:** A user initiates a transaction (e.g., a balance check or payment) from a mobile device or web browser.
- **Mechanism:** The request routes through **AWS WAF** (Web Application Firewall), which inspects the payload for malicious code, SQL injection, and DDoS indicators.
- **Routing:** Clean traffic passes directly to the **Amazon API Gateway** positioned within the public boundaries of the primary Virtual Private Cloud (VPC).

Step 2: Traffic Load Balancing & Ingress Control (Anchor 6)

- **Action:** The API Gateway validates tokens, manages rate limiting, and forwards the clean request down the pipeline.
- **Mechanism:** The request strikes the **Application Load Balancer (ALB)** acting as the Kubernetes ingress controller.

- **Routing:** The ALB terminates the external HTTPS connection and distributes the incoming traffic across healthy target pods running inside the private subnet layer.

Step 3: Containerized Microservice Processing (Anchor 7)

- **Action:** The request hits the targeted microservice pod (Payment, Balance, or Account) running on **Amazon EKS** (Elastic Kubernetes Service).
- **Mechanism:** The container checks **Amazon ElastiCache** to instantly retrieve frequently read, non-sensitive session data. This saves valuable database compute time.
- **State Management:** For complex or multi-step transactions, the container publishes an asynchronous event directly to **Amazon MSK** (Managed Streaming for Apache Kafka) to guarantee message durability and decoupled processing.

Step 4: Secure Legacy Hybrid Integration (Anchor 2, 3, & 5)

- **Action:** The transaction requires formal validation against the legacy system or cryptographic authorization from a Hardware Security Module (HSM).
- **Mechanism:** The EKS service securely communicates via private **VPC Endpoints** into the **AWS Transit Gateway**.
- **Routing:** Traffic travels across a dedicated, low-latency **AWS Direct Connect** private line directly into the bank's on-premise data center. The request hits the **Request Gateway** to query the on-premise HSM for PIN/cryptographic validation and updates the core banking registry.

Step 5: Data Persistence & Synchronization (Anchor 8 & 9)

- **Action:** The on-premise system confirms validation, sending a success signal back across the Direct Connect pipeline to AWS.
- **Mechanism:** The Amazon EKS container commits the final, audited transaction record into the **Amazon RDS Primary** database.
- **Resiliency:** The data is immediately and synchronously replicated to an **RDS Standby** instance across a separate Availability Zone to protect

against hardware failure. It is also asynchronously replicated to an **RDS Replica** node for heavy downstream analytical read requests.

Step 6: Out-of-Band Notification & Auditing (Anchor 10, 11, 12, & 14)

- **Action:** The transaction finishes, triggering confirmation workflows and generating telemetry.
- **Mechanism:** Workers inside the **Amazon EC2 Auto Scaling** group ingest success events from the Amazon MSK broker to fire off out-of-band **Short Message Service (SMS)** mobile notifications to the end user.
- **Observability:** Throughout this entire journey, structured runtime execution logs, API latency metrics, and system exceptions are streamed silently to **Amazon CloudWatch** and **Amazon OpenSearch Service** for real-time operations auditing and threat detection.

Security Architecture

This architecture implements a **Zero Trust** security model designed to achieve strict compliance with financial regulations such as **PCI-DSS (Payment Card Industry Data Security Standard)** and **SOC 2**. Security boundaries are strictly enforced at every layer of the system.

1. Network Segregation and Perimeter Defense (Anchors 4 & 6)

- **Ingress Protection: AWS WAF** mitigates Layer 7 injection attacks and volumetric DDoS threats at the public boundary.
- **Microservices Isolation:** The **Amazon EKS** cluster runs entirely within **Private Subnets**. These subnets have no direct routing paths to or from the public internet.
- **Control Plane Security:** Traffic entering the EKS environment must traverse the **Application Load Balancer (ALB) Ingress**, which enforces TLS termination using modern, secure cipher suites.

2. Cryptographic Isolation & Tokenization (Anchor 13)

- **Data-at-Rest Encryption:** All transactional databases (**Amazon RDS**) and caching tiers (**Amazon ElastiCache**) enforce AES-256 storage-level encryption managed via **AWS KMS** (Key Management Service).
- **Envelope Encryption:** High-value payloads (e.g., Primary Account Numbers) are tokenized immediately upon ingress. The system utilizes envelope encryption: data is encrypted using local Data Encryption Keys (DEKs), which are wrapped by a customer-managed root Key Encryption Key (KEK) inside AWS KMS.
- **On-Premise HSM Linkage (Anchor 3):** To satisfy regulatory requirements for pinning transactions, AWS KMS configurations interface with the bank's on-premise **Hardware Security Modules (HSMs)** through secure, dedicated **AWS Direct Connect** tunnels.

3. Identity & Least Privilege Access Management

- **Service-to-Service Authorization:** The system enforces strict separation of duties using **AWS IAM** combined with **EKS IAM Roles for Service Accounts (IRSA)**.
- **Granular Scoping:** Pods running the payment microservice are granted explicit, narrow permissions to write to specific RDS tables, while onboarding pods are completely blocked from accessing payment data.
- **Secret Storage:** Database credentials, API tokens, and TLS certificates are never hardcoded inside container images. Instead, they are dynamically injected at runtime via an external secrets operator backed by AWS Secrets Manager.

Disaster Recovery (DR) Plan

To support critical financial operations, this system uses a **Warm Standby / Pilot Light** hybrid model targeting strict service level objectives:

- **Recovery Point Objective (RPO):** < 1 Minute (Maximum acceptable data loss)
- **Recovery Time Objective (RTO):** < 15 Minutes (Maximum acceptable downtime)

1. Multi-Availability Zone Failover (High Availability)

- **Database Resiliency (Anchor 9): Amazon RDS** is configured in a Multi-AZ deployment. Writes to the primary instance are synchronously mirrored to an **RDS Standby** instance in a separate AZ. If the primary zone suffers a physical outage, a seamless, automatic DNS failover occurs in under 60 seconds with zero data loss.
- **Compute Elasticity:** EKS worker nodes are evenly provisioned across three separate AZs. If an entire AZ drops offline, the **EC2 Auto Scaling** groups automatically provision replacement pods in the remaining healthy zones to handle incoming transaction loads.

2. Cross-Region Replica Topology (Disaster Recovery)

For catastrophic regional failures, a secondary, completely independent AWS backup region is maintained:

- **Asynchronous Replication: Amazon RDS Read Replicas** continuously stream transactional logs asynchronously across AWS regions.
- **Container Image Portability (Anchor 15): Amazon ECR** (Elastic Container Registry) repositories are cross-region replicated. This ensures that the exact same production-certified Kubernetes manifests and docker images are instantly available to deploy in the backup region.

3. Failover Execution Workflow

1. **Detection:** Route 53 health checks monitor the primary region's ingress ALB. If consecutive failures are logged, an automated DevOps alert is triggered.
2. **Traffic Diversion:** Global DNS routing (Amazon Route 53) shifts client transaction API endpoints toward the secondary recovery region's entry point.
3. **Database Promotion:** The cross-region RDS Read Replica is promoted to a standalone primary read/write database instance.
4. **Compute Scaling:** The secondary EKS cluster scales up its workloads from minimum "pilot light" capacity to full production levels to absorb the incoming transaction traffic.

Concerns

The Synchronous Hybrid Bottleneck (The "Distributed Monolith" Risk)

- **The Flaw:** Step 4 of the transaction flow shows the Amazon EKS cluster communicating synchronously through the AWS Transit Gateway and Direct Connect to the on-premise Core Banking System and HSM to validate transactions.
- **The Risk:** This creates a tight runtime coupling. If the bank's on-premise data center experiences latency spikes or network jitter over the Direct Connect line, the EKS container threads will back up waiting for a response. This can quickly exhaust the container connection pool, leading to a cascading failure that brings down the cloud-native front end.
- **The Fix:** Move to an asynchronous, event-driven pattern for non-instant operations using an orchestration pattern (like AWS Step Functions) to manage long-running transactional states safely without blocking active threads.

2. Microservices Tightly Coupled to a Shared Database

- **The Flaw:** The diagram shows multiple distinct application sub-domains (payment, balance, Account, and onboarding) deployed across Availability Zones, but they all appear to converge downstream into a single, centralized Amazon RDS Primary instance.
- **The Risk:** This violates a core microservices tenet: *Database-per-Service*. Sharing a single relational database database creates tight data-schema coupling. A database migration or schema update for the onboarding service risks breaking the mission-critical payment service. Furthermore, a single database instance represents a massive single point of failure (SPOF) and a severe I/O bottleneck during transaction spikes.
- **The Fix:** Decouple the data layer. Transition transient data like balances to a high-throughput NoSQL database (like Amazon DynamoDB), and keep Amazon RDS strictly scoped to the payment service ledger.

3. OpenSearch Service Inside the Critical Path Workflow

- **The Flaw:** While Amazon OpenSearch Service is excellent for log aggregation and search analytics, it is a heavy, stateful, and expensive cluster component that requires specialized maintenance.

- **The Risk:** If the architecture relies on OpenSearch for operational auditing or real-time transaction history checks inside the user-facing API workflow, it introduces high operational complexity. OpenSearch clusters can become sluggish during massive indexing spikes (such as a high-volume trading day), impacting overall transaction response times.
- **The Fix:** Use OpenSearch strictly out-of-band for telemetry and troubleshooting. Real-time transaction history should be served via key-value stores like Amazon ElastiCache or DynamoDB, with cold logs archived cheaply in Amazon S3.

4. Excessive Cross-AZ Data Transfer Costs

- **The Flaw:** The architecture relies heavily on cross-AZ traffic, showing multiple microservices, caching layers (ElastiCache), and messaging brokers (Amazon MSK) communicating constantly across different Availability Zones.
- **The Risk:** In AWS, you pay data transfer fees for every gigabyte of data that crosses an Availability Zone boundary. For a high-frequency payment platform processing millions of transactions a day, cross-AZ traffic charges can easily become one of the largest lines on the monthly cloud bill.
- **The Fix:** Implement **Topology-Aware Routing** in Kubernetes to ensure that a container pod preferentially talks to dependencies (like a database replica or cache node) residing within its *same* physical Availability Zone, minimizing cross-AZ data hops.

5. Multi-Region DR Realities (RTO vs. RPO Clashes)

- **The Flaw:** The Disaster Recovery section outlines an asynchronous cross-region replication model for Amazon RDS to achieve low-cost cross-region backup capabilities.
- **The Risk:** During a true regional failover event, forcing an asynchronous database replica to become the primary instance introduces a high risk of **split-brain scenarios** or data loss for transactions that were in-flight right before the crash. For a bank, even a 30-second window of lost transactional data requires days of manual financial reconciliation.
- **The Fix:** If the business truly mandates near-zero RPO/RTO for disaster recovery, the relational database must be upgraded to **Amazon Aurora**

Global Database, which leverages storage-based physical replication to bring cross-region replication lag down to under a second.

6. **Blast Radius Vulnerability in the Transit Gateway**

- **Flaw:** Production payment pipelines, administrative services, and legacy core integration traffic all route through a single, shared configuration layer inside a single AWS Transit Gateway.
- **Risk:** A minor human misconfiguration or automated deployment error within the central Transit Gateway route tables can instantly sever connectivity between the cloud Amazon EKS cluster and the on-premise core bank, triggering an immediate, total platform outage.
- **Fix:** Isolate your infrastructure into a multi-account structure using **AWS Organizations**. Deploy dedicated Transit Gateway route tables per environment (Development, Staging, Production) to strictly contain network changes and limit the blast radius of any infrastructure misconfiguration.

7. **Distributed Tracing Blind Spot Across Microservices**

- **Flaw:** The platform relies heavily on decoupled, asynchronous Amazon MSK message hops to pass transactional payloads between individual payment, balance, and Account microservice pods.
- **Risk:** Without a unified tracing framework, tracking down a single stuck or corrupted payload across multiple boundary changes becomes an operational nightmare. Debugging simple latency issues turns into a massive time sink, leaving teams blind during live production outages.
- **Fix:** Embed the **AWS Distro for OpenTelemetry** as a sidecar proxy across all containerized workloads. Inject a unique trace ID into transaction headers right at the Amazon API Gateway boundary to gain end-to-end visual tracing across every container and Kafka broker hop.

8. **Single Point of Failure (SPOF) in Outbound Network Gateways**

- **Flaw:** The private subnet routing topology points to a single, centralized NAT Gateway to manage outbound requests to external third-party endpoints, such as credit bureaus and card networks.
- **Risk:** If the physical Availability Zone hosting that single NAT Gateway experiences a hardware failure, all outbound communication from the

entire platform is instantly paralyzed, regardless of how healthy the EKS clusters are in other zones.

- **Fix:** Provision independent, dedicated **NAT Gateways in every separate Availability Zone** within the public subnets. Configure your private subnet route tables to use only their local zone's NAT Gateway to eliminate cross-zone network dependencies.

9. **Cache Stampede and Database Failure Vulnerability**

- **Flaw:** The architecture relies on an Amazon ElastiCache tier to offload frequent data checks (like user balance lookups) from the primary database without any fallback logic for synchronized cache expiration.
- **Risk:** During peak traffic events, if a highly accessed cache key expires, thousands of incoming transaction pods will simultaneously register a cache miss and strike the Amazon RDS primary instance at the exact same millisecond. This sudden "cache stampede" can instantly max out database CPU and crash the ledger.
- **Fix:** Update the application code inside the EKS pods to enforce distributed locking (via Redis) during a cache miss, ensuring only a single worker pod goes to rebuild the data. Additionally, implement probabilistic early expiration algorithms to refresh hot data in the background before the key officially dies.

10. **ConfigMap Secret Sprawl and Exposure**

- **Flaw:** High-value environment configurations—such as database passwords, API keys for external financial networks, and Kafka encryption credentials—are managed natively within standard Kubernetes ConfigMaps or default Secrets.
- **Risk:** Default Kubernetes secrets are only base64-encoded, not securely encrypted by default. This exposes sensitive production credentials to anyone with basic cluster read access, violating core **PCI-DSS** and **SOC 2** access control principles.
- **Fix:** Eradicate hardcoded or static cluster secrets by deploying the **AWS Secrets Manager Kubernetes Secrets Store CSI Driver**. This pattern injects sensitive credentials directly into the container pod's volatile memory space at runtime and automates rotation schedules without requiring application restarts.

Architectural Recommendations

1. **Decouple Hybrid Network Dependencies**

- **Action:** Shift from synchronous runtime dependencies to an **asynchronous saga orchestration pattern**.
- **Implementation:** Utilize **AWS Step Functions** to orchestrate multi-step transaction states (e.g., hold, validate, settle). If the on-premise Core Banking System or HSM experiences latency spikes over the Direct Connect line, the cloud ingress tier remains unaffected, processing incoming requests into durable message queues instead of exhausting container thread pools.

2. Enforce Database-per-Service Isolation

- **Action:** Break down the monolithic shared database bottleneck into decoupled data stores.
- **Implementation:** Isolate the payment, Account, and onboarding data layers. Transition high-throughput, transient state queries (like balance checks) to **Amazon DynamoDB** for single-digit millisecond performance at scale. Restrict **Amazon RDS** strictly to the core payment transaction ledger to prevent schema coupling and eliminate a massive single point of failure (SPOF).

3. Optimize Cross-AZ Data Traffic Costs

- **Action:** Reduce high-frequency, cross-Availability Zone networking fees.
- **Implementation:** Configure **Topology-Aware Hints** within the **Amazon EKS** cluster. This forces Kubernetes services to route traffic preferentially to container pods, Amazon ElastiCache instances, and database replicas located within the *same* physical Availability Zone, slashing cross-AZ data transfer line items on the AWS invoice.

4. Harden Disaster Recovery Tolerances

- **Action:** Bridge the financial data-loss gap during catastrophic regional outages.
- **Implementation:** Migrate standard Amazon RDS instances to **Amazon Aurora Global Database**. Aurora utilizes dedicated storage-level replication to achieve a cross-region replication lag of less than one

second, minimizing the risk of split-brain data discrepancies and ensuring a near-zero Recovery Point Objective (RPO) for critical ledger entries.

5. Isolate Observability Infrastructure

- **Action:** Remove search clusters from the synchronous transaction runtime loop.
- **Implementation:** Constrain **Amazon OpenSearch Service** strictly to an out-of-band operational role for security log analytics and debugging. Use high-performance key-value databases to power active customer-facing transaction histories, while pushing raw application logs into **Amazon S3** via Firehose before indexing into OpenSearch.

6. **Implement Network Segmentation via Multi-Account AWS Organizations**
   - **Action:** Restructure the single-account infrastructure topology to minimize the architectural blast radius.
   - **Implementation:** Transition the platform into a multi-account environment managed by **AWS Organizations**. Isolate core workloads into distinct AWS accounts (e.g., *Network-Ingress Account*, *Core-Compute Account*, and *Data-Persistence Account*). Connect these environments using **AWS Transit Gateway**, enforcing strictly separated, non-overlapping route tables to completely segregate production payment lines from administrative systems.

7. **Introduce Distributed Tracing via AWS X-Ray and OpenTelemetry**
   - **Action:** Eliminate operational visibility blind spots across decoupled asynchronous microservice boundaries.
   - **Implementation:** Deploy the **AWS Distro for OpenTelemetry (ADOT)** as a sidecar container daemon across all **Amazon EKS** application pods. Configure the **Amazon API Gateway** to inject a unique X-Amzn-Trace-Id header at the public boundary, and configure the application code to pass this context through **Amazon MSK** message headers. This allows **AWS X-Ray** to map and trace transaction lifecycles across container networks and Kafka brokers in real time.**Deploy Multi-AZ High-Availability NAT Gateways**

8. **Deploy Multi-AZ High-Availability NAT Gateways**

- **Action:** Mitigate the single point of failure (SPOF) present in the outbound network gateway layer.
- **Implementation:** Provision independent, localized **NAT Gateways inside every active Availability Zone (AZ)** within the public subnet architecture. Update the private subnet route tables of each respective AZ (rtb-private-az1, rtb-private-az2, etc.) to route outbound 0.0.0.0/0 traffic exclusively through its zone-specific NAT Gateway. This ensures that an unexpected outage in a single AWS data center cannot disrupt outbound traffic in other healthy zones.

9. **Mitigate Stampedes via Cache Locking and Probabilistic Early Expiry**

- **Action :** Protect the primary SQL ledger from high-volume database crashes caused by cache stampedes.
- **Implementation:** Integrate a distributed locking library (such as *Redlock*) within the microservices application code using **Amazon ElastiCache (Redis)**. When a cache miss occurs on a high-concurrency key (like a user balance), the pod must acquire a short-lived mutex lock, ensuring only a single worker thread queries **Amazon RDS** to rebuild the cache while other identical requests wait and retry.

10. **Enforce Enterprise Secret Injection with AWS Secrets Manager & Vault**

- **Action:** Eliminate static, base64-encoded Kubernetes secrets to comply with strict PCI-DSS access controls.
- **Implementation:** Install the **AWS Secrets Manager Secrets Store CSI Driver** onto the **Amazon EKS** cluster. Configure IAM Roles for Service Accounts (IRSA) to grant individual pods read-only access to specific secret ARNs. Mount these secrets as local, volatile volumes (tmpfs) directly into the container's memory space at launch, allowing **AWS Secrets Manager** to handle routine password rotations automatically without requiring application or pod restarts.